push_all.sh — user guide

Revision: 1 **Last modified:** 2026-06-22T00:00:00Z

Companion documentation (§11.4.18) for `scripts/push_all.sh`.

Overview

`push_all.sh` pushes a branch to **every** configured upstream remote of the helix_ota main repository. The project fans out to four mirrors — `github` (primary) + `gitlab` + `gitflic` + `gitverse` — per the multi-upstream-push norm (§2.1). The script is designed to run **detached** (`nohup … &`) per §11.4.88 so that the calling `commit_all.sh` releases its commit lock the moment the local commit is durable, and the slow per-mirror network push proceeds in the background.

Key properties:

- **Per-remote serialization** via a portable lock so two concurrent invocations never push the same remote simultaneously, while different remotes still push in parallel.
- **Retry with exponential backoff** per remote (default 3 attempts, doubling delay).
- **Pre-push fetch** of every remote (§11.4.71) with a per-remote timeout so an unreachable mirror does not hang the run.
- **Honest exit accounting** (§11.4.1) — exit 1 + an explicit `N remote(s) NOT confirmed pushed` line whenever any remote was skipped, hard-failed, or left in an unknown state. The script NEVER reports `All upstreams pushed` unless every remote is genuinely confirmed.
- **GitFlic packfile phased-push fallback** for remotes that reject a push exceeding their packfile size limit.

This script does NOT force-push under any circumstance — it is a fast-forward `git push <remote> <branch>` only, consistent with the absolute no-force-push mandate (§11.4.113).

Prerequisites

- A git repository with the four upstream remotes configured (`github`, `gitlab`, `gitflic`, `gitverse`). These are installed by the constitution submodule's `install_upstreams.sh` (§11.4.36).
- `git` (>= 2.30) and `bash` 4+ on `PATH`.
- `timeout` (coreutils / `gtimeout`) on `PATH` for the per-remote fetch guard.
- Network reachability to the mirrors (unreachable mirrors are handled gracefully — see Edge cases).

- `flock` is **optional**: it is used when present (Linux) and transparently replaced by a portable `mkdir` lock when absent (macOS/BSD) — see Internal behaviour.

Usage examples

```
# Push the current branch to all four upstreams
bash scripts/push_all.sh

# Explicit branch
bash scripts/push_all.sh main
bash scripts/push_all.sh --branch main

# Override retry count + initial backoff delay
bash scripts/push_all.sh --branch main --retries 5 --delay 10

# Detached background push (the §11.4.88 invocation commit_all.sh
        uses)
nohup bash scripts/push_all.sh > /tmp/push.log 2>&1 &
disown

# Quiet mode (log file still written)
bash scripts/push_all.sh --quiet
```

Inputs

Flag	Default	Meaning
`-b, --branch NAME`	current branch (`git branch --show-current`, else `main`)	Branch to push
`-r, --retries N`	`3`	Max push attempts per remote
`-d, --delay SECS`	`5`	Initial retry delay; doubles each attempt (exponential backoff)
`-l, --log PATH`	auto: `qa-results/push_failures/<UTC-timestamp>_push.log`	Log file path
`-q, --quiet`	off	Suppress stdout (the log file is still written)
`-h, --help`	—	Show usage and exit 0

A bare positional argument is interpreted as the branch name (so `push_all.sh main` and `push_all.sh --branch main` are equivalent). The remote list is fixed in the script: `github gitlab gitflic gitverse`.

Outputs

Output	Meaning
Per-remote push to the four mirrors	the actual work
Log file (`qa-results/push_failures/<ts>_push.log` by default)	per-remote attempt/status trail + a `PUSH SUMMARY` block
stdout `[PUSH]` / `[PUSH-OK]` / `[PUSH-WARN]` / `[PUSH-ERR]` lines	live per-remote status (suppressed by `--quiet`)
Exit code	`0` only when ALL remotes confirmed (OK or PHASED); `1` if any remote is NOT confirmed pushed

Per-remote status is one of: `OK` (pushed), `PHASED` (packfile-limited remote handed off to the bundle path), `SKIPPED_LOCKED` (another push held the lock), `FAILED` (exhausted retries), `UNKNOWN` (never reached).

Edge cases

- **Unreachable / slow remote (pre-push fetch).** Each pre-push fetch is wrapped in `timeout 15`; a timeout or fetch error logs a `PUSH-WARN` and the run continues to the push phase rather than hanging.
- **`flock` absent (macOS/BSD).** `flock(1)` is Linux-only and is NOT present on macOS/BSD (§11.4.67/§11.4.81). The historical bug was that the script invoked `flock` unconditionally — on macOS this produced `flock: command not found`, the lock was silently skipped on every remote, and the summary then falsely reported success. The fix: the script now uses an atomic `mkdir`-based lock (`.git/.push.<remote>.lock.d`) that works on every POSIX host, and only prefers `flock` when it is genuinely present.
- **Stale lock from a dead process.** When the `mkdir` lock directory already exists, the script reads the holder PID from `.git/.push.<remote>.lock.d/pid` and, if that process is gone (`kill -0` fails), breaks the stale lock and re-acquires it. If a live process still holds the lock the remote is marked `SKIPPED_LOCKED` (not pushed — and therefore counted against the honest exit).
- **GitFlic packfile-size limit.** When a push is rejected for exceeding the remote's packfile limit (matched on `Pack exceeds the limit` / `packfile … limit` / `unpacker error`), the script falls back to `_phased_push`: it fetches the remote branch, computes how many commits are ahead, and if behind generates a `git bundle … --all` under `.git/gitflic_bundle_<ts>/` for manual sync, marking the remote `PHASED`.
- **Push failure (other).** Each non-packfile failure is retried up to `--retries` times with exponential backoff; on exhaustion the remote is marked `FAILED` and counted in `TOTAL_FAILURES`.
- **Partial / all-skipped run — honest exit (§11.4.1).** A remote counts as "pushed" ONLY if its status is `OK` or `PHASED`. Any `SKIPPED_LOCKED`, `FAILED`, or `UNKNOWN` remote increments `NOT_CONFIRMED`; if `NOT_CONFIRMED > 0` (or any hard failure) the script logs `N remote(s) NOT confirmed pushed (M hard-failed)` and **exits 1**. It prints `All upstreams pushed` and exits 0 only when every remote is confirmed — preventing the prior false-success bluff where every remote was skipped yet the run exited 0.

Internal behaviour

1. `set -euo pipefail`; resolve `SCRIPT_DIR` + `PROJECT_ROOT`, parse flags, resolve the branch, and auto-generate the log path under `qa-results/push_failures/` if `--log` was not supplied.
2. **Pre-push fetch (§11.4.71)** — for each remote, `timeout 15 git fetch <remote> --prune`; timeouts/errors log a warning and continue.
3. **Per-remote push** (`push_one_remote`) for each of the four remotes:
   - **Lock acquisition.** If `flock` is present, acquire an exclusive non-blocking `flock` on fd 9 over `.git/.push.<remote>.lock`; if held, mark `SKIPPED_LOCKED`. If `flock` is absent, `mkdir .git/.push.<remote>.lock.d` atomically; on contention, break the lock only if the recorded holder PID is dead, else mark `SKIPPED_LOCKED`. The lock is released via a `RETURN` trap.
   - **Retry loop** up to `--retries`: run `git push <remote> <branch>`. On rc 0 → `OK`. On a packfile-limit match → `_phased_push` → `PHASED`. On other failure → warn, sleep `delay`, double `delay`, retry. On exhaustion → `FAILED` + increment `TOTAL_FAILURES`.
4. **Summary** — append a `PUSH SUMMARY` block to the log and print a per-remote status line to stdout.
5. **Honest accounting (§11.4.1)** — count `NOT_CONFIRMED` remotes (anything not `OK`/`PHASED`); if any failure or unconfirmed remote exists, log the `NOT confirmed pushed` line and exit 1; otherwise log `All upstreams pushed` and exit 0.

Related scripts

- `scripts/commit_all.sh` — the canonical project commit wrapper. It commits locally, releases its commit lock immediately, then spawns `push_all.sh` **detached** so the slow multi-mirror push runs in the background (§11.4.88). `push_all.sh` is the push half of that two-stage flow.
- `constitution/install_upstreams.sh` — configures the four upstream remotes this script fans out to (§11.4.36); a prerequisite for `push_all.sh` to have anything to push to.

Last verified date

2026-06-22 — documented against the script as committed (read, not inferred): portable `mkdir` per-remote lock with `flock`-when-present preference (§11.4.67/ §11.4.81), dead-PID stale-lock break, exponential- backoff retry, GitFlic packfile phased-push fallback, and honest exit-1 accounting on any unconfirmed remote (§11.4.1).